Design and Analysis of Algorithms – Assignment 2 (CO2)

Source Code: solution_code.py (Q1–Q7, including graph code)

```
#!/usr/bin/env python3
"""
Design and Analysis of Algorithms - Assignment 2 (CO2)
Brute-force and exhaustive-search experiments for Q1-Q7.

Run:  python3 solution_code.py
Output: raw_data/*.csv, charts/*.png, results.json (all numbers used in the report)

Operation counts are the primary evidence. Execution time is secondary and
will differ from machine to machine.
"""
import csv
import json
import math
import os
import random
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(HERE, "raw_data")
CHARTS = os.path.join(HERE, "charts")
os.makedirs(DATA, exist_ok=True)
os.makedirs(CHARTS, exist_ok=True)

plt.rcParams["font.family"] = "serif"
plt.rcParams["font.serif"] = ["Times New Roman", "Liberation Serif", "DejaVu Serif"]
plt.rcParams["font.size"] = 10

RES = {}


def save_csv(name, header, rows):
    with open(os.path.join(DATA, name), "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(header)
        w.writerows(rows)


def timed(fn, repeats=3):
    """Return (result, best time in seconds)."""
    best = float("inf")
    res = None
    for _ in range(repeats):
        t0 = time.perf_counter()
        res = fn()
        best = min(best, time.perf_counter() - t0)
    return res, best
```

```
datasets["D5 Reverse sorted"] = base[::-1]
datasets["D6 All equal scores"] = [5000] * n

algos = [("Selection", selection_sort), ("Bubble", bubble_sort), ("Insertion", insertion_sort)]
rows = []
table = {}
for dname, data in datasets.items():
    for aname, fn in algos:
        (out, c, m), t = timed(lambda: fn(data), repeats=3)
        assert out == sorted(data)
        rows.append([dname, aname, n, c, m, round(t * 1000, 3)])
        table[(dname, aname)] = (c, m, t * 1000)
save_csv("q1_sorting_results.csv",
         ["dataset", "algorithm", "n", "comparisons", "swaps_or_shifts", "time_ms"], rows)
RES["q1_main"] = [
    {"dataset": r[0], "algo": r[1], "n": r[2], "comps": r[3], "moves": r[4], "ms": r[5]}
    for r in rows
]

# batch-size sweep: n = 1000, sorted prefix + random batch appended
sweep = []
for b in [1, 10, 50, 100, 250, 500]:
    data = sorted(rng.sample(range(1000, 100000), n - b)) + rng.sample(range(1000, 100000), b)
    row = {"batch": b, "pct": 100 * b / n}
    for aname, fn in algos:
        _, c, m = fn(data)
        row[aname + "_comps"] = c
        row[aname + "_moves"] = m
    sweep.append(row)
save_csv("q1_batch_sweep.csv",
         ["batch_size", "percent_new", "sel_comps", "sel_swaps", "bub_comps", "bub_swaps",
          "ins_comps", "ins_shifts"],
         [[r["batch"], r["pct"], r["Selection_comps"], r["Selection_moves"], r["Bubble_comps"],
           r["Bubble_moves"], r["Insertion_comps"], r["Insertion_moves"]] for r in sweep])
RES["q1_sweep"] = sweep

# scaling check on random data
scal = []
for nn in [250, 500, 1000, 2000]:
    data = rng.sample(range(1, 10 * nn), nn)
    row = {"n": nn}
    for aname, fn in algos:
        _, c, m = fn(data)
        row[aname + "_comps"] = c
        row[aname + "_moves"] = m
    scal.append(row)
save_csv("q1_scaling_random.csv",
         ["n", "sel_comps", "sel_swaps", "bub_comps", "bub_swaps", "ins_comps", "ins_shifts"],
         [[r["n"], r["Selection_comps"], r["Selection_moves"], r["Bubble_comps"], r["Bubble_moves"],
           r["Insertion_comps"], r["Insertion_moves"]] for r in scal])
RES["q1_scaling"] = scal

# chart
names = list(datasets.keys())
short = ["D1\nRandom", "D2\nSorted", "D3\nEdge", "D4\n95% sorted", "D5\nReverse", "D6\nEqual"]
fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
for ax, idx, title in [(axes[0], 0, "Comparisons"), (axes[1], 1, "Swaps / shifts")]:
    w = 0.27
    for k, (aname, _) in enumerate(algos):
        vals = [max(table[(d, aname)][idx], 0.5) for d in names]
        ax.bar([i + (k - 1) * w for i in range(len(names))], vals, w, label=aname)
    ax.set_yscale("log")
    ax.set_xticks(range(len(names)))
```

```
        ax.set_xticklabels(short, fontsize=8)
        ax.set_title(title + " (n = 1000, log scale)")
        ax.set_ylabel("count")
        ax.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(CHARTS, "q1_sorting.png"), dpi=170)
    plt.close()


# ============================================================================
# Q2  Sequential search
# ============================================================================
def make_records(n, rng):
```

```
n = 10000
m = 50
tests = []

# T1: best-type: first pattern character never occurs in the text
t1 = "".join(rng.choice("abcdefghijklmnopqrstuvwxyz") for _ in range(n))
p1 = "#" + "".join(rng.choice("abcdefghijklmnopqrstuvwxyz") for _ in range(m - 1))
# T2: engineered worst-type
t2 = "A" * n
p2 = "A" * (m - 1) + "B"
# T3: binary alphabet random text, random pattern (own choice)
t3 = "".join(rng.choice("01") for _ in range(n))
p3 = "".join(rng.choice("01") for _ in range(m))
```

```
# ============================================================================
# Q5  Brute-force convex hull
# ============================================================================
def line_coeffs(p, q):
    """Line through p=(x1,y1), q=(x2,y2):  a*x + b*y = c."""
    a = q[1] - p[1]
    b = p[0] - q[0]
    c = p[0] * q[1] - p[1] * q[0]
    return a, b, c
```

```
conv = []
for n in [10, 20, 40, 80, 160]:
    pts = [(x, x * x) for x in range(n)]
    h, t, _ = hull_bruteforce(pts, True)
    conv.append({"n": n, "hull": len(h), "tests": t})
RES["q5_convex_curve"] = conv

# visualisation
fig, axes = plt.subplots(1, 2, figsize=(10, 4.4))
for ax, (pts, hull, title) in zip(
        axes, [(set1, tests[0]["hull"], "Set 1: 30 random points"),
               (set2, tests[1]["hull"], "Set 2: 5x5 grid with duplicates")]):
    ax.scatter([p[0] for p in pts], [p[1] for p in pts], s=18, color="#4c78a8", zorder=3)
```

```
W = [12, 7, 11, 8, 9]
V = [24, 13, 23, 15, 16]
CAP = 26
rows = []
for mask in range(32):
    sel = [i for i in range(5) if mask >> i & 1]
    wt = sum(W[i] for i in sel)
    val = sum(V[i] for i in sel)
    rows.append({"set": ["P%d" % (i + 1) for i in sel], "w": wt, "v": val, "feasible": wt <= CAP})
truth = max((r for r in rows if r["feasible"]), key=lambda r: r["v"])
bv, bvec, cnt = exhaustive(W, V, CAP)
RES["q6_small"] = {"W": W, "V": V, "cap": CAP, "all": rows, "truth_v": truth["v"], "truth_set": truth["set"],
                   "algo_v": bv, "algo_set": ["P%d" % (i + 1) for i in range(5) if bvec[i]] if False else None,
```

```
    (bv, sel, cnt), t = timed(lambda: branch_and_bound(w, v, cap), repeats=1)
    assert bv == dp_knapsack(w, v, cap)
    adv.append({"n": n, "nodes": cnt["nodes"], "orig_nodes": 2 ** (n + 1) - 1, "time": t,
                "ratio": cnt["nodes"] / (2 ** (n + 1) - 1)})
RES["q7_adv"] = adv
save_csv("q7_adversarial_instance.csv", ["n", "redesign_nodes", "original_nodes", "fraction_of_original", "time_s"],
         [[a["n"], a["nodes"], a["orig_nodes"], round(a["ratio"], 3), round(a["time"], 4)] for a in adv])

fig, ax = plt.subplots(figsize=(6.4, 4))
ns = [e["n"] for e in res] + [b["n"] for b in big]
ax.semilogy([e["n"] for e in res], [e["orig_nodes"] for e in res], "k--o", label="original (2^(n+1) - 1)")
ax.semilogy(ns, [e["new_nodes"] for e in res] + [b["nodes"] for b in big], "s-", label="redesign (mean of 5)")
ax.semilogy([a["n"] for a in adv], [a["nodes"] for a in adv], "^:", label="redesign, adversarial instance")
```